Coding & Solution

Date	04 July 2026
Team ID	
Project Name	Rising Waters – Flood Prediction System
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of the implemented code and the overall solution.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/Laharisrikotipalli/RisingWaters
Programming Language(s)	Python (backend), HTML/CSS/JavaScript (frontend)
Framework(s) / Libraries Used	Flask, Flask-SQLAlchemy, Flask-Login, Flask-WTF, WTForms, Authlib (Google OAuth), python-dotenv, Gunicorn (production server)
Key Features Implemented	10-field prediction form with server-side validation (WTForms), user registration and login (email/password and Google OAuth via Authlib), CSRF-protected forms, prediction history stored per user in a database (SQLAlchemy), Joblib model + scaler inference, animated risk gauge result pages, light/dark theme toggle, flash-message feedback, custom 404/500 error pages, Render deployment config
Pending / Incomplete Features	Password-reset emails, rate limiting on login/register, a JSON /api/predict endpoint, and live weather API auto-fill for the prediction form
Setup / Run Instructions	cp .env.example .env (set SECRET_KEY) → pip install -r requirements.txt → python app.py → open http://127.0.0.1:5000

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	User input is validated before being processed	Yes
6	The application runs without critical errors	Yes

S.No	Criteria	Status (Yes / No)
7	Code is committed to a version control repository	Yes

Additional Notes / Comments:

Since the initial submission, the application was extended with a full authentication layer (Flask-Login, Flask-WTF, Authlib for Google sign-in), a SQLAlchemy database storing users and their prediction history, and server-side form validation replacing the earlier raw request.form access. The /predict route now requires a logged-in user, validates all 10 fields with WTForms, and saves every prediction so it can be reviewed or deleted from a new /history page. Custom 404 and 500 error pages and flash-message feedback were also added, and the app still degrades gracefully — it keeps running with prediction disabled — if the model or scaler files fail to load.

Supporting Screenshots

Space reserved below for screenshots of the running application and repository.

[Insert screenshot: application running locally]

[Insert screenshot: GitHub repository / commit history]